

Arquitetura de Software — Cidade Inteligente ESP32 + FreeRTOS

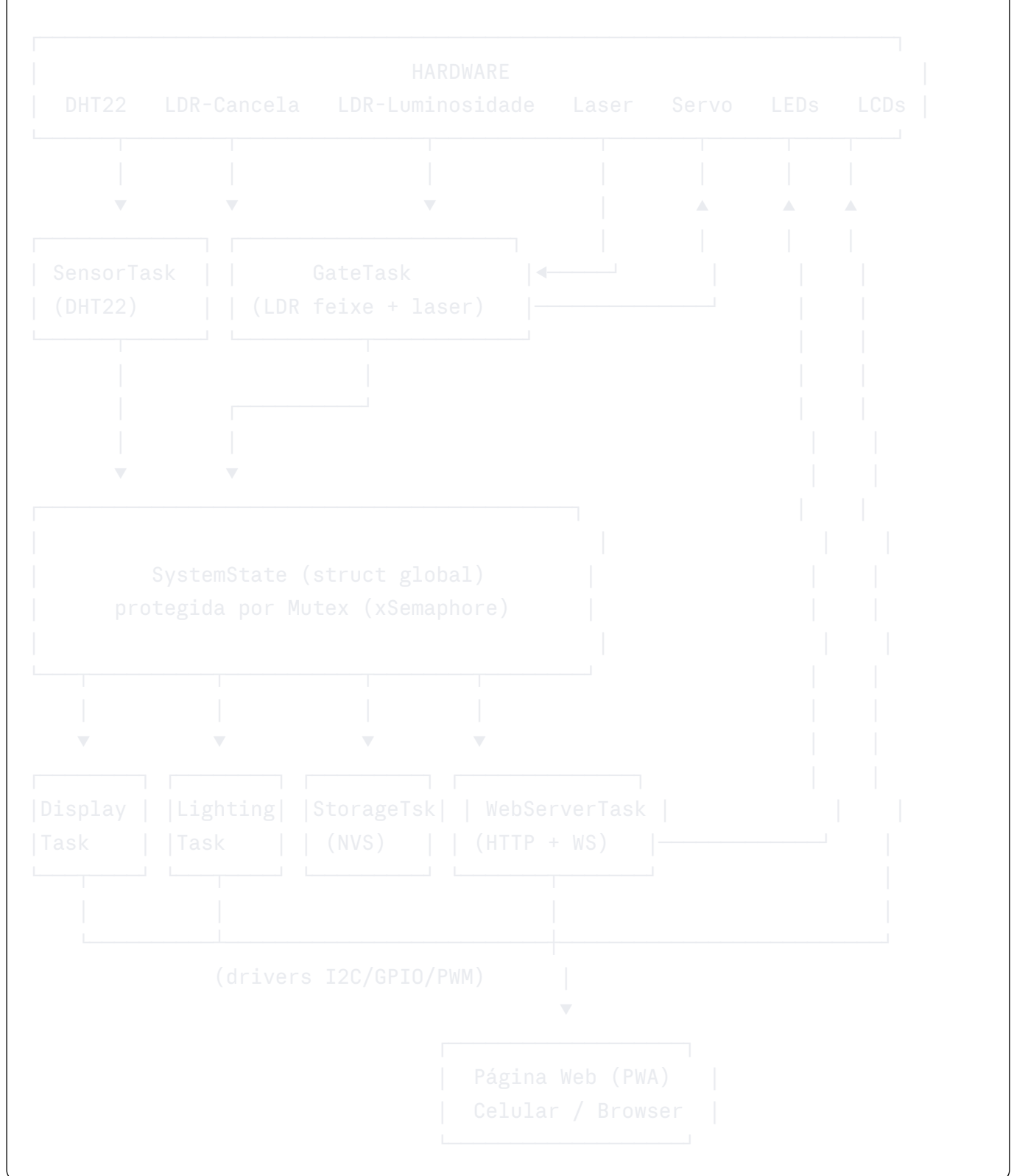
Documento de Arquitetura de Sistema Embarcado Versão 1.0

Sumário

1. Diagrama de blocos do sistema
 2. Tarefas FreeRTOS
 3. Comunicação entre tarefas
 4. Estrutura de dados global
 5. Máquina de estados da cancela
 6. Arquitetura Web
 7. Organização de pastas
 8. Fluxograma completo (boot → operação)
 9. Escalabilidade futura
-

1. Diagrama de Blocos do Sistema

A arquitetura segue o padrão **produtor-consumidor com estado central protegido**, onde tarefas de aquisição (sensores) escrevem em uma estrutura de dados global (`SystemState`), e tarefas de apresentação/atuação (displays, servidor web, cancela, iluminação) leem/escrevem nessa mesma estrutura de forma sincronizada.



Fluxo lógico:

- `SensorTask` e `GateTask` são **produtoras** de dados (temperatura, umidade, luminosidade, estado da cancela).
- `SystemState` é o **hub central**, acessado com Mutex.
- `DisplayTask`, `LightingTask`, `WebServerTask` e `StorageTask` são **consumidoras**, cada uma lendo o que precisa.
- `WebServerTask` também é produtora indireta: comandos vindos do celular (abrir cancela, mudar nome da cidade) escrevem de volta em `SystemState` via fila de comandos.

2. Tarefas FreeRTOS

Prioridades no ESP32 (Arduino/ESP-IDF) variam de 0 (idle) a 24/25 (máx). Uso uma escala prática de 0 a 5, compatível com `configMAX_PRIORITIES` padrão do Arduino-ESP32 (geralmente 25, mas raramente se usa mais que 5-6 níveis num projeto didático).

Tarefa	Responsabilidade	Prioridade	Período	Stack sugerida	CPU esperada
GateTask	Ler LDR do feixe laser, detectar interrupção, controlar servo (abrir/fechar), aplicar máquina de estados, temporizador de auto-fechamento	4 (alta)	20 ms (polling do LDR)	3072 B	Baixa, picos curtos no acionamento do servo
SensorTask	Ler DHT22 (temp/umidade), validar leitura, escrever em <code>SystemState</code>	3	2000 ms (DHT22 não deve ser lido mais rápido que ~1x/s, recomenda-se 2s)	2560 B	Muito baixa (~1% — maior parte do tempo bloqueada)
LightingTask	Ler LDR de luminosidade ambiente, calcular PWM proporcional, aplicar <code>ledcWrite</code> nos LEDs	3	200 ms	2048 B	Muito baixa
DisplayTask	Atualizar LCD 1 (nome da cidade / mensagens de status) e LCD 2 (temperatura/umidade), com fila de mensagens para eventos assíncronos	2	500 ms (ou orientado a evento via fila)	2560 B	Baixa (I2C é rápido)
WebServerTask	Servidor HTTP assíncrono (AsyncWebServer) + WebSocket, expõe endpoints REST, recebe comandos, envia atualizações	2	Orientada a evento (callbacks assíncronos) + broadcast a cada 1000 ms	4096–6144 B	Variável, picos durante requisições, baixa em repouso
StorageTask	Persistir nome da cidade e configurações (tempo de auto-fechamento da cancela) em NVS/SPIFFS, evitando	1 (baixa)	Orientada a evento (fila de "salvar"), debounce de 3000 ms	2560 B	Muito baixa, esporádica

Tarefa	Responsabilidade	Prioridade	Período	Stack sugerida	CPU esperada
	gravações excessivas (debounce)				
WatchdogTask (opcional, recomendada)	Monitorar heap livre, status do Wi-Fi, alimentar Task Watchdog Timer, reconectar Wi-Fi se necessário	1	5000 ms	2048 B	Desprezível

Justificativa das prioridades:

- `GateTask` recebe a maior prioridade porque envolve **segurança física** (evitar que a cancela feche sobre um veículo, resposta rápida à interrupção do laser). É a tarefa mais "tempo-crítica" do sistema.
- `SensorTask` e `LightingTask` ficam no nível intermediário: precisam rodar com regularidade, mas toleram atraso de dezenas/centenas de ms sem problema.
- `DisplayTask` e `WebServerTask` são de **apresentação**, toleram maior latência (o usuário não percebe 200-500ms de atraso num display ou numa página web).
- `StorageTask` tem a prioridade mais baixa porque grava em memória não-volátil (operação relativamente lenta e que não deve bloquear o resto do sistema); além disso, sofre debounce para não desgastar a flash com escritas repetidas.

Nota de projeto: no ESP32 (dual-core), é recomendável fixar `WebServerTask` e `GateTask`/`SensorTask` em cores diferentes usando `xTaskCreatePinnedToCore()`.

Sugestão:

- **Core 0 (PRO_CPU):** Wi-Fi/WebServerTask (o stack Wi-Fi já roda nativamente aqui)
- **Core 1 (APP_CPU):** SensorTask, GateTask, LightingTask, DisplayTask, StorageTask

3. Comunicação entre Tarefas

Mecanismo	Onde é usado	Por quê
Mutex ((<code>SemaphoreHandle_t</code>), tipo (<code>xSemaphoreCreateMutex</code>))	Proteger o acesso à struct (<code>SystemState</code>) (leitura/ escrita por múltiplas tarefas)	Mutex é a escolha correta (em vez de semáforo binário comum) porque suporta priority inheritance , evitando inversão de prioridade quando uma tarefa de baixa prioridade (ex: (<code>StorageTask</code>)) segura o lock enquanto uma de alta prioridade ((<code>GateTask</code>)) tenta acessá-lo.
Queue ((<code>xQueueCreate</code>)) — (<code>commandQueue</code>)	(<code>WebServerTask</code>) → (<code>GateTask</code>) / (<code>SystemState</code>) (comandos: abrir cancela, fechar cancela, trocar nome da cidade)	Comandos vindos da web são eventos discretos e assíncronos. Uma fila permite desacoplar totalmente quem gera o comando (HTTP callback) de quem executa (<code>GateTask</code>), sem risco de perda se chegarem rápido — cada item é processado em ordem (FIFO).
Queue — (<code>displayMsgQueue</code>)	Qualquer tarefa → (<code>DisplayTask</code>)	Permite que eventos (ex: "Cancela Aberta", "Erro no sensor") sejam exibidos imediatamente no LCD sem que (<code>DisplayTask</code>) precise "adivinhar" que algo mudou — desacopla produtores de mensagens do consumidor.
Queue — (<code>storageQueue</code>)	(<code>WebServerTask</code>) / (<code>GateTask</code>) → (<code>StorageTask</code>)	Envia apenas o dado que precisa ser persistido (ex: novo nome da cidade). A (<code>StorageTask</code>) aplica debounce antes de gravar na flash.
Task Notification ((<code>xTaskNotify</code>) / (<code>xTaskNotifyFromISR</code>))	ISR do LDR do feixe laser → (<code>GateTask</code>)	Task Notifications são mais leves e rápidas que filas/semáforos (não alocam buffer, endereçamento direto à TCB da tarefa). Ideais para o caso "interrupção do feixe → acordar (<code>GateTask</code>) imediatamente", que é o evento mais tempo-crítico do sistema. Usa-se (<code>xTaskNotifyFromISR</code>) para respeitar o contexto de interrupção (ISR-safe).

Mecanismo	Onde é usado	Por quê
Event Group (<code>xEventGroupCreate</code>)	Sincronização de estado de sistema: <code>WIFI_CONNECTED_BIT</code> , <code>SNTP_SYNCED_BIT</code> , <code>STORAGE_LOADED_BIT</code>	Várias tarefas (ex: <code>WebServerTask</code> , <code>DisplayTask</code>) precisam esperar múltiplas condições antes de operar plenamente (ex: só iniciar o servidor HTTP depois que o Wi-Fi conectar E as configurações forem carregadas do NVS). Event Groups permitem <code>xEventGroupWaitBits()</code> com múltiplos bits e lógica AND/OR, o que uma fila ou semáforo simples não faria de forma elegante.
Semáforo binário (<code>xSemaphoreCreateBinary</code>)	Opcional: sincronizar <code>GateTask</code> com fim de movimento do servo (se usar servo com feedback)	Sinalização simples de "evento ocorreu", sem necessidade de payload de dados — mais leve que uma fila quando não há dado a transportar (mas normalmente substituído por Task Notification, mais eficiente ainda).

Regra geral de projeto adotada:

- Dado que muda com frequência e é lido por várias tarefas → protegido por **Mutex** dentro da struct global.
- Evento discreto/comando pontual → **Queue**.
- Sinal simples vindo de ISR, sem dado → **Task Notification**.
- Espera por múltiplas condições de sistema → **Event Group**.

4. Estrutura de Dados Global

C

```

// include/SystemState.h
#pragma once
#include <stdint.h>
#include <stdbool.h>
#include "freertos/FreeRTOS.h"
#include "freertos/semphr.h"

#define CITY_NAME_MAX_LEN 32

typedef enum {
    GATE_CLOSED = 0,
    GATE_OPENING,
    GATE_OPEN,
    GATE_CLOSING,
    GATE_ERROR
} GateState_t;

typedef enum {
    WIFI_DISCONNECTED = 0,
    WIFI_CONNECTING,
    WIFI_CONNECTED,
    WIFI_AP_FALLBACK // modo AP local se STA falhar (recurso didático útil)
} WifiState_t;

typedef struct {
    // --- Ambiente ---
    float temperature; // °C
    float humidity; // %
    bool dhtValid; // última leitura foi válida?

    // --- Iluminação ---
    uint16_t ambientLightRaw; // leitura bruta do LDR (0-4095)
    uint8_t lightingPWM; // duty cycle aplicado aos LEDs (0-255)

    // --- Cancela ---
    GateState_t gateState;
    bool gateManualOverride; // true = comando manual ativo
    uint32_t gateAutoCloseMs; // tempo configurável de auto-fechamento
    uint32_t gateLastTriggerTick; // instante da última interrupção do feixe

    // --- Cidade ---
    char cityName[CITY_NAME_MAX_LEN];

    // --- Conectividade ---
    WifiState_t wifiState;
    char ipAddress[16];

    // --- Mutex de proteção ---

```



```

        SemaphoreHandle_t mutex;
    } SystemState_t;

extern SystemState_t g_systemState;

// API de acesso seguro (wrapper – recomendado ao invés de acesso direto)
bool SystemState_Init(void);
bool SystemState_Lock(TickType_t timeout);
void SystemState_Unlock(void);

```

c

```

// src/SystemState.c
#include "SystemState.h"

SystemState_t g_systemState;

bool SystemState_Init(void) {
    g_systemState.mutex = xSemaphoreCreateMutex();
    strncpy(g_systemState.cityName, "Cidade Modelo", CITY_NAME_MAX_LEN);
    g_systemState.gateState = GATE_CLOSED;
    g_systemState.gateAutoCloseMs = 5000; // padrão: 5s
    g_systemState.wifiState = WIFI_DISCONNECTED;
    return g_systemState.mutex != NULL;
}

bool SystemState_Lock(TickType_t timeout) {
    return xSemaphoreTake(g_systemState.mutex, timeout) == pdTRUE;
}

void SystemState_Unlock(void) {
    xSemaphoreGive(g_systemState.mutex);
}

```

Padrão de uso em qualquer tarefa (exemplo em `SensorTask`):

c

```

if (SystemState_Lock(pdMS_TO_TICKS(50))) {
    g_systemState.temperature = newTemp;
    g_systemState.humidity    = newHum;
    g_systemState.dhtValid    = true;
    SystemState_Unlock();
} else {
    // timeout no lock: log de erro, não trava o sistema
}

```

Por que essa abordagem:

- Um único Mutex global é **simples e suficiente** para o porte de um projeto didático (menos overhead cognitivo, menos risco de deadlock por múltiplos locks).
- O timeout no `xSemaphoreTake` evita bloqueio infinito (falha "graciosa" em vez de travar o sistema).
- Encapsular acesso em funções (`SystemState_Lock/Unlock`) evita que alguém esqueça de liberar o mutex — boa prática de engenharia.
- Para um projeto mais avançado, poderia-se particionar em **mutexes por domínio** (ambiente, cancela, cidade) para reduzir contenção, mas isso é otimização prematura aqui.

5. Máquina de Estados da Cancela

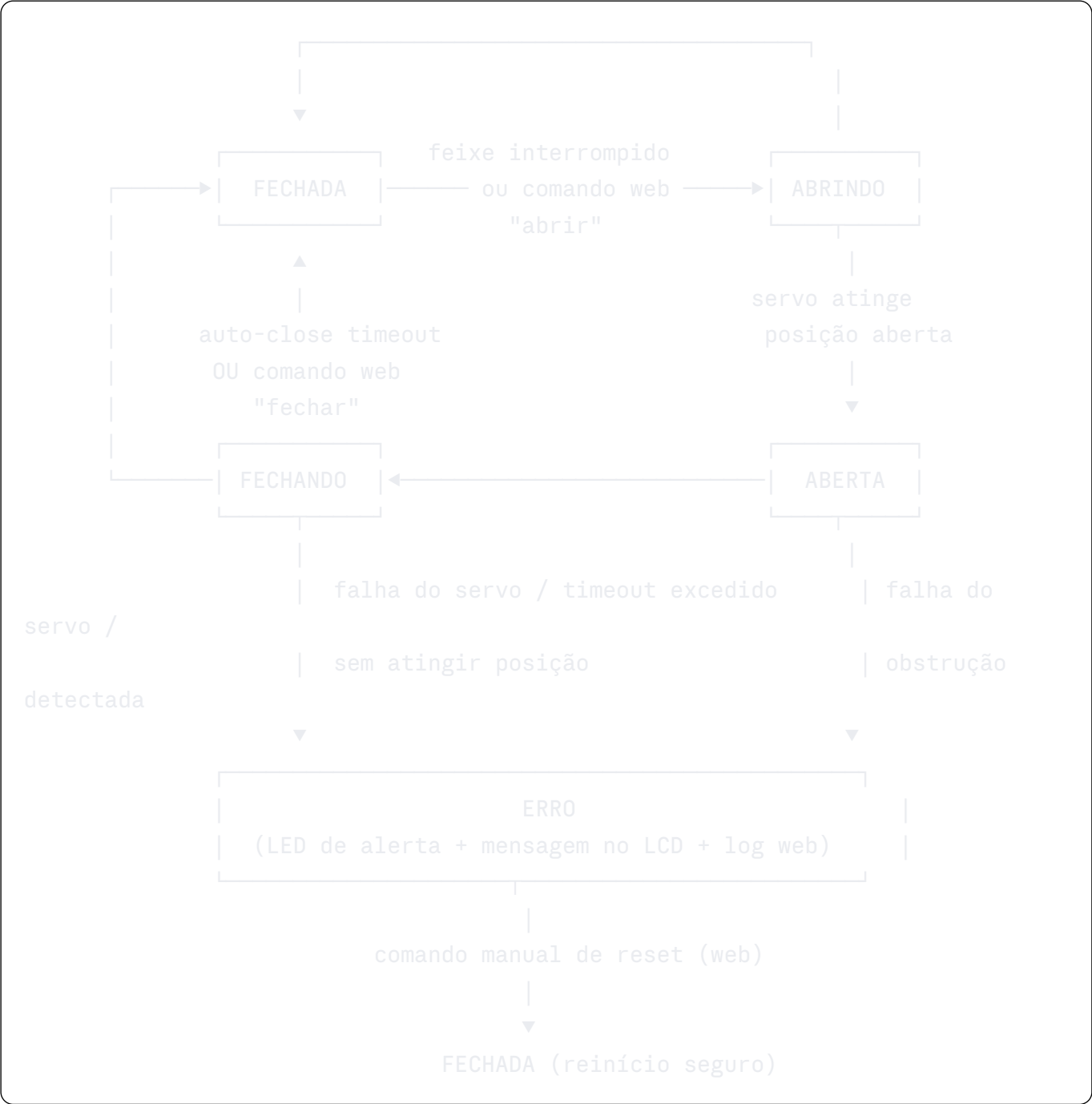


Tabela de transições:

Estado atual	Evento	Próximo estado	Ação
FECHADA	Feixe interrompido (LDR) OU comando web "abrir"	ABRINDO	Aciona servo, inicia timeout de movimento
ABRINDO	Servo atinge posição aberta (tempo estimado ou feedback)	ABERTA	Zera <code>gateLastTriggerTick</code> , inicia contagem de auto-close
ABRINDO	Timeout de movimento excedido	ERRO	Interrompe PWM do servo, sinaliza erro
ABERTA	<code>gateAutoCloseMs</code> expira OU comando web "fechar"	FECHANDO	Aciona servo em sentido inverso
ABERTA	Feixe interrompido novamente	ABERTA (renova timer)	Reinicia contagem de auto-close (evita fechar em cima de algo passando)
FECHANDO	Servo atinge posição fechada	FECHADA	Libera PWM do servo (economia de energia)
FECHANDO	Feixe interrompido durante o fechamento	ABRINDO	Segurança: reabre imediatamente
FECHANDO	Timeout de movimento excedido	ERRO	Interrompe PWM, sinaliza erro
ERRO	Comando manual "reset" via web	FECHADA	Reinicializa servo na posição segura

Essa máquina roda **dentro da** `GateTask`, avaliada a cada ciclo (20 ms), com transições disparadas por Task Notification (ISR do LDR), pela fila `commandQueue` (comandos web) ou por comparação de `xTaskGetTickCount()` contra o timeout configurado.

6. Arquitetura Web

Stack recomendada

- **ESPAsyncWebServer** + **AsyncTCP** (biblioteca assíncrona, não bloqueia tarefas do FreeRTOS como o `WebServer.h` síncrono faria).
- **WebSocket** (`AsyncWebSocket`) para atualização em tempo real (temperatura, umidade, estado da cancela, luminosidade) — evita polling HTTP constante do celular.
- HTTP REST tradicional para comandos pontuais (abrir/fechar cancela, mudar nome da cidade).

Endpoints REST

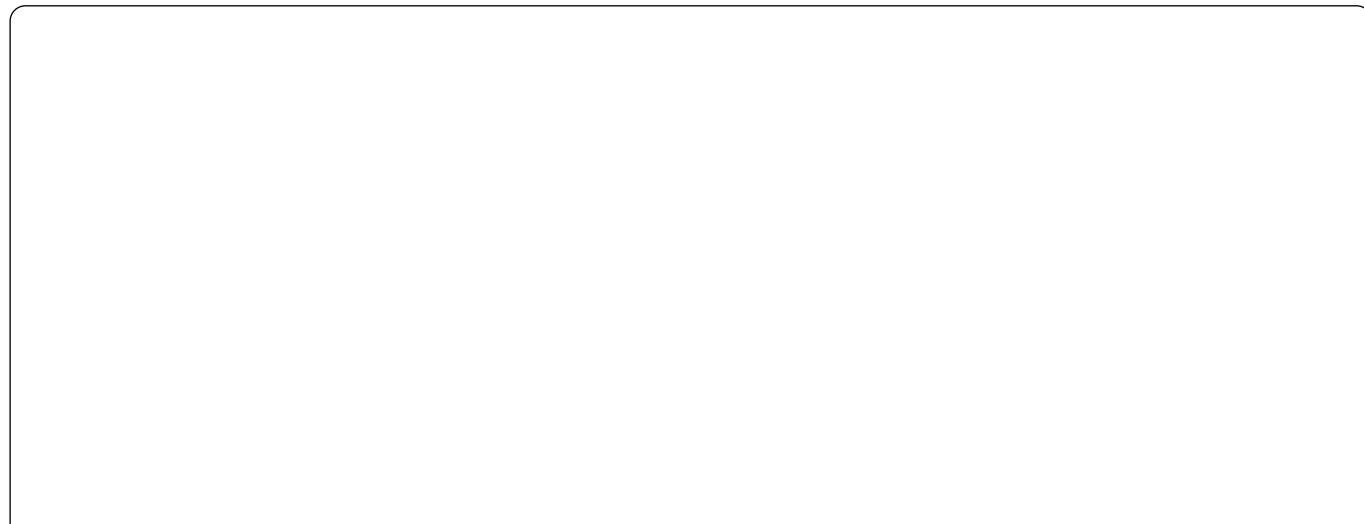
Método	Endpoint	Função	Corpo/Resposta
GET	/api/status	Retorna snapshot completo do estado	JSON (ver abaixo)
POST	/api/gate/open	Comando manual: abrir cancela	{"result": "ok"}
POST	/api/gate/close	Comando manual: fechar cancela	{"result": "ok"}
POST	/api/city/name	Altera nome da cidade	body: {"name": "Nova Cidade"}
GET	/api/city/name	Consulta nome atual	{"name": "Cidade Modelo"}
WS	/ws	Canal WebSocket para push periódico	JSON a cada 1s (broadcast)

Exemplo de JSON de resposta (GET /api/status ou broadcast via WS)

json

```
{
  "temperature": 27.4,
  "humidity": 61.2,
  "dhtValid": true,
  "ambientLight": 812,
  "lightingPWM": 145,
  "gateState": "OPEN",
  "cityName": "Cidade Modelo",
  "wifiState": "CONNECTED",
  "ip": "192.168.1.42"
}
```

Fluxo assíncrono





Por que WebSocket + REST híbrido, e não só polling AJAX:

- Polling AJAX (`setInterval`) fazendo `fetch()` a cada 1s) funciona, mas gera overhead de handshake HTTP repetido e não escala bem se houver vários clientes conectados simultaneamente na maquete (didaticamente comum: vários alunos com celular).
- WebSocket mantém uma conexão persistente e permite `broadcast` eficiente — todos os celulares recebem a atualização simultaneamente com uma única serialização de JSON.
- Comandos (ações pontuais) continuam em REST porque são raros e HTTP simples é suficiente e mais fácil de debugar (`curl`, Postman).

Front-end (`app.js`) — esqueleto de integração

javascript

```
const ws = new WebSocket(`ws://${location.host}/ws`);

ws.onmessage = (event) => {
  const data = JSON.parse(event.data);
  document.getElementById("temp").innerText = data.temperature.toFixed(1) + " °C";
  document.getElementById("hum").innerText = data.humidity.toFixed(1) + " %";
  document.getElementById("light").innerText = data.ambientLight;
  document.getElementById("gateState").innerText = data.gateState;
};

function openGate() { fetch("/api/gate/open", { method: "POST" }); }
function closeGate() { fetch("/api/gate/close", { method: "POST" }); }

function setCityName(name) {
  fetch("/api/city/name", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name })
  });
}
```

7. Organização de Pastas

atualização de LCM.

(compilados/embarcados via SPIFFS/LittleFS ou como PROGMEM, conforme escolha)

```
/include
```

```
SystemState.h
```

→ Definição da struct global, enums de estado, API de lock/unlock – único ponto de verdade do sistema.

```
Config.h
```

```
via WiFiManager),
```

→ Pinos, constantes de timing, SSID/senha (ou

```
mágicos".
```

tamanhos de fila e stack – centraliza "números

```
Tasks.h
```

→ Protótipos das funções de entrada de cada task e handles globais (TaskHandle_t, QueueHandle_t etc).

Racional da organização:

- `/drivers` isola completamente o hardware — se trocar o modelo do LCD ou do sensor, só se edita um arquivo, sem tocar na lógica de negócio.
- `/tasks` contém a lógica de orquestração FreeRTOS, sempre chamando funções de `/drivers`, nunca acessando registradores diretamente.
- `/include/SystemState.h` é o contrato entre todas as tasks — qualquer novo dado do sistema nasce ali.
- `/web` fica separado do firmware C++, podendo até ser desenvolvido/testado isoladamente num navegador antes de ser embarcado no SPIFFS/LittleFS.

8. Fluxograma Completo (Boot → Operação Normal)


```

|         → envia mensagem para displayMsgQueue se mudou
|
| LightingTask: a cada 200ms lê LDR → calcula PWM → ledcWrite
|
| DisplayTask: bloqueada em xQueueReceive(displayMsgQueue)
|               OU timeout de 500ms → refresca LCDs
|
| WebServerTask: aguarda requisições HTTP assíncronas
|               + timer de 1000ms faz broadcast via WebSocket
|
| StorageTask: bloqueada em xQueueReceive(storageQueue)
|              → debounce 3000ms → grava no NVS
|
|-----|
|
|         ▼ (evento assíncrono a qualquer momento)
| ISR do LDR do feixe → xTaskNotifyFromISR(GateTask) → acorda GateTask
|
|         ▼
| Comando via app web → HTTP handler → xQueueSend(commandQueue,...)
|
|         ▼
| (sistema permanece nesse laço de eventos indefinidamente,
|  com WatchdogTask opcional monitorando saúde geral)

```

9. Escalabilidade Futura

A arquitetura foi desenhada para crescer **sem quebrar o núcleo existente**, graças ao desacoplamento via `SystemState` + filas. Caminhos de expansão:

9.1 MQTT

- Adicionar uma `MqttTask` (prioridade similar à `WebServerTask`) que **assina** `SystemState` da mesma forma que `DisplayTask` faz — publica em tópicos como `cidade/temperatura`, `cidade/cancela/estado`.
- Comandos MQTT recebidos (`cidade/cancela/comando`) alimentam a **mesma** `commandQueue` já usada pela web — nenhuma mudança na `GateTask`.
- Broker sugerido para ensino: Mosquitto local ou HiveMQ Cloud (grátis para protótipos).

9.2 Dashboard em Nuvem

- `MqttTask` ou uma `CloudSyncTask` dedicada envia snapshots periódicos para um backend (Node-RED, ThingsBoard, Firebase, AWS IoT Core).
- Como `SystemState` já é a fonte única da verdade, basta mais um "consumidor" — não se toca nas tasks de sensoriamento/atuação.

9.3 Sensores adicionais (ex: qualidade do ar, chuva, nível de ruído)

- Cada novo sensor ganha sua própria task leve (`AirQualityTask`, etc.) seguindo o

mesmo padrão: ler → `SystemState_Lock()` → gravar campo novo → `Unlock()`.

- `SystemState_t` cresce com novos campos; `/api/status` e o WebSocket JSON são atualizados de forma aditiva (compatível com clientes antigos, que ignoram campos novos).

9.4 Câmeras (ex: ESP32-CAM dedicado)

- Recomenda-se um **segundo ESP32-CAM** como nó independente (streaming MJPEG consome muita CPU/RAM e disputaria recursos com o Wi-Fi stack principal).
- Esse nó se comunica com o ESP32 principal via MQTT (ex: publica "movimento detectado" no LDR de vigilância) ou expõe seu próprio endpoint HTTP, referenciado por um `<img>` na página web central.

9.5 Controle de tráfego (semáforos, múltiplas cancelas)

- Reaproveita-se o padrão `GateTask` como um "template de atuador com máquina de estados": cria-se `TrafficLightTask` seguindo a mesma estrutura (estado, fila de comando, Task Notification de sensores de presença).
- Se o número de atuadores crescer muito, considerar migrar de "uma task por atuador" para **um gerenciador único de atuadores** (`ActuatorManagerTask`) com uma tabela de atuadores registrados, evitando explosão de tasks/stacks no ESP32 (RAM é o recurso mais escasso).

9.6 Considerações gerais de escalabilidade

- **RAM é o limitador real** no ESP32 (~320KB SRAM). Ao adicionar tasks, monitorar `uxTaskGetStackHighWaterMark()` e o heap livre (`esp_get_free_heap_size()`), idealmente reportado pela `WatchdogTask`.
- Para crescimento além do ESP32 único, a arquitetura já é "pronta para distribuir": qualquer módulo pode migrar para um ESP32 satélite comunicando-se por MQTT/ESP-NOW, mantendo o padrão de `SystemState` como contrato de dados.

Resumo Executivo

Aspecto	Decisão de Projeto
Concorrência	6-7 tasks FreeRTOS, prioridades por criticidade física > apresentação > persistência
Estado	Struct única (<code>SystemState_t</code>) protegida por Mutex com priority inheritance
Eventos discretos	Queues (<code>commandQueue</code> , <code>displayMsgQueue</code> , <code>storageQueue</code>)
Eventos de ISR	Task Notifications (mais leve, ISR-safe)
Sincronização de boot	Event Group (<code>WIFI_CONNECTED_BIT</code> , etc.)
Web	ESPAsyncWebServer + WebSocket (tempo real) + REST (comandos)
Persistência	NVS via <code>Preferences.h</code> , com debounce para preservar ciclos de escrita da flash
Segurança da cancela	Máquina de estados explícita com estado de ERRO e recuperação manual

Este documento serve como referência de implementação — cada seção pode ser desenvolvida incrementalmente (ex: primeiro `SensorTask` + `DisplayTask`, depois `GateTask`, depois a camada web), validando a integração a cada etapa antes de avançar.